



MASTER INTELLIGENCE ARTIFICIELLE
TRAVAUX DIRIGÉS — MACHINE LEARNING

TD N° 05

Arbres de Décision & K-Nearest Neighbors (KNN)

Durée : 3 heures	Niveau : Intermédiaire
Datasets : BankChurners + synthétique 2D	Outils : sklearn, numpy, matplotlib, graphviz

Objectifs du TD

- **Calculer manuellement** l'entropie, l'impureté de Gini et le gain d'information
- Comprendre la **construction récursive** d'un arbre (algorithme ID3/CART)
- **Visualiser** un arbre de décision complet avec sklearn et graphviz
- Analyser l'effet de **max_depth** et **min_samples_leaf** sur le sur-apprentissage
- Implémenter KNN et trouver le **K optimal** par courbe Elbow
- Démontrer expérimentalement la **nécessité de normaliser** pour KNN
- **Comparer** Arbre (interprétable) vs KNN (non-paramétrique) sur le même dataset
- Comprendre la **malédiction de la dimensionnalité** — limites du KNN

Pr. BEN LAHMAR EL Habib · Semestre 2024–2025



Contexte et Rappels Théoriques

Ce TD met en parallèle deux algorithmes aux philosophies opposées : l'**Arbre de Décision** — interprétable, basé sur des règles SI/ALORS — et le **KNN** — non-paramétrique, basé sur la distance. Les deux ont des forces complémentaires selon le contexte.

Rappel de cours — Arbres de Décision — Construction et Critères de Split

- **Principe** : partitionner récursivement l'espace des features par des questions binaires (test $x_i \leq \text{seuil}$?)
- **Algorithmes** : ID3 (entropie, données discrètes), C4.5 (entropie, continu + manquants), CART (Gini, sklearn)
- **Critère d'entropie** : $H(S) = -\sum p_i \log_2(p_i)$ Nœud pur : $H=0$ / Nœud aléatoire : $H=1$ (binaire)
- **Impureté de Gini** : $G(S) = 1 - \sum p_i^2$ Nœud pur : $G=0$ / Maximum : $G=0.5$ (binaire)
- **Gain d'information** : $IG(S,A) = H(S) - \sum (|S_v|/|S|) \cdot H(S_v)$ → choisir l'attribut maximisant IG
- **Avantages** : interprétable, pas de normalisation, gère les valeurs manquantes
- **Inconvénients** : instable (haute variance), sur-apprentissage facile sans élagage

Rappel de cours — K-Nearest Neighbors — Algorithme et Propriétés

- **Algorithme lazy** : aucun entraînement proprement dit — tout le calcul se fait à la prédiction
- **Principe** : classer un nouveau point x par vote majoritaire de ses K plus proches voisins
- **Distance Euclidienne** : $d(x,y) = \sqrt{\sum (x_i - y_i)^2}$ — sensible aux différences d'échelle → normalisation obligatoire
- **Choix de K** : petit K → frontière complexe (sur-apprentissage) / grand K → frontière lisse (sous-apprentissage)
- **Malédiction de la dimensionnalité** : en haute dimension, tous les points deviennent équidistants → KNN perd sa pertinence
- **Complexité de prédiction** : $O(n \cdot d)$ — lent sur grands datasets
- **Quand l'utiliser** : petit dataset, peu de features, pas de relation linéaire évidente

Critère	Arbre de Décision	KNN
Entraînement	Construction de l'arbre $O(n \cdot d \cdot \log n)$	Aucun — lazy learner
Prédiction	$O(\text{profondeur arbre}) \approx O(\log n)$	$O(n \cdot d)$ — coûteux
Normalisation	Non requise	Obligatoire
Interprétabilité	Très élevée — règles lisibles	Nulle — boîte noire



Critère	Arbre de Décision	KNN
Hyperparamètre clé	max_depth, min_samples_leaf	K (nombre de voisins)
Haute dimension	Se dégrade (overfitting)	Se dégrade fortement (malédiction)

Préparez l'environnement de travail :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import (DecisionTreeClassifier, export_text,
                          plot_tree, export_graphviz)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import (train_test_split, cross_val_score,
                                     validation_curve)
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, f1_score,
                             confusion_matrix, ConfusionMatrixDisplay)
from sklearn.pipeline import Pipeline
import warnings ; warnings.filterwarnings('ignore')

# Dataset BankChurners (encodé depuis TD 2)
raw = pd.read_csv('BankChurners.csv')
data = raw[raw.columns[:-2]].copy()
data['churn'] = (data['Attrition_Flag']=='Attrited Customer').astype(int)

# Encodage rapide (résumé TD 2)
data['Gender'] = data['Gender'].map({'M':1, 'F':0})
data['Marital_Status'] =
data['Marital_Status'].map({'Single':0, 'Married':1, 'Divorced':2, 'Unknown':-1})
edu_map = {'Unknown':0, 'Uneducated':1, 'High School':2, 'College':3, 'Graduate':4, 'Post-
Graduate':5, 'Doctorate':6}
inc_map = {'Unknown':0, 'Less than $40K':1, '$40K - $60K':2, '$60K - $80K':3, '$80K -
$120K':4, '$120K +':5}
data['Education_Level'] = data['Education_Level'].map(edu_map)
data['Income_Category'] = data['Income_Category'].map(inc_map)
data = pd.get_dummies(data, columns=['Card_Category'], prefix='card',
drop_first=True, dtype=int)

feat_cols = [c for c in data.columns if c not in
['Attrition_Flag', 'churn', 'CLIENTNUM']]
X = data[feat_cols] ; y = data['churn']
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
print(f'Train: {X_tr.shape} | Test: {X_te.shape}')
```



Partie 1 — Calculs Manuels : Entropie, Gini et Gain d'Information

Durée estimée : 35 minutes | Comprendre les formules avant de les confier à sklearn — la vraie maîtrise de l'algorithme.

Entropie de Shannon : $H(S) = - \sum p_i \times \log_2(p_i)$

Gini : $G(S) = 1 - \sum p_i^2$ | **Gain :** $IG(S,A) = H(S) - \sum |S_v|/|S| \times H(S_v)$

Q 1.1

Calcul manuel d'entropie et d'impureté de Gini

Consigne

Calculez manuellement l'entropie et l'impureté de Gini pour les distributions suivantes, puis vérifiez avec NumPy.

Nœud	Classe 0 (Existing)	Classe 1 (Attrited)	Entropie	Gini
A — Pur	10	0	???	???
B — Équilibré	5	5	???	???
C — Réel (83.9/16.1)	839	161	???	???
D — Après split feat1	600	40 / 239	???	???

```
def entropie(classes):  
    """Calcule l'entropie de Shannon d'une distribution."""  
    total = sum(classes)  
    if total == 0: return 0  
    H = 0  
    for n in classes:  
        if n > 0:  
            p = n / total  
            H -= p * np.log2(p)  
    return H  
  
def gini(classes):  
    """Calcule l'impureté de Gini."""  
    total = sum(classes)  
    if total == 0: return 0  
    return 1 - sum((n/total)**2 for n in classes)
```



```
# — Calculer pour chaque nœud du tableau —————
noeuds = {
    'A - Pur':      [10, 0],
    'B - Équilibré': [5, 5],
    'C - Réel':     [839, 161],
    'D - Gauche':   [600, 40],
    'D - Droite':    [239, 121],
}

print(f'{"Nœud":20s} {"Entropie":>10s} {"Gini":>8s}')
print('-' * 45)
for nom, dist in noeuds.items():
    H = entropie(dist)
    G = gini(dist)
    print(f'{"nom":20s} {"H":>10.4f} {"G":>8.4f}')
```

Questions d'analyse

1. Pour le nœud A (pur) : l'entropie et le Gini valent-ils 0 ? Expliquez pourquoi un nœud pur est l'**objectif** de l'arbre de décision.
2. Pour le nœud B (équilibré 50/50) : quelle est la valeur maximale de l'entropie (binaire) ? Et du Gini ? Ces valeurs correspondent-elles aux formules théoriques ?
3. Entropie vs Gini : pour le nœud C (83.9/16.1), laquelle des deux mesures donne une valeur plus haute ? En pratique, les deux critères donnent-ils des arbres très différents ?

Q 1.2

Gain d'information — choisir le meilleur attribut de split

Consigne

Calculez le gain d'information pour **Total_Trans_Ct** et **Months_Inactive_12_mon**. Comparez avec le gain calculé par sklearn (feature_importances_).

```
def gain_information(y, y_left, y_right, criterion='entropy'):
    """Calcule le gain d'information d'un split."""
    n = len(y) ; n_l = len(y_left) ; n_r = len(y_right)

    if criterion == 'entropy':
        fn = entropie
        dist_fn = lambda arr: [sum(arr==0), sum(arr==1)]
    else:
        fn = gini
        dist_fn = lambda arr: [sum(arr==0), sum(arr==1)]
```



```
H_parent = fn(dist_fn(y))
H_left   = fn(dist_fn(y_left))
H_right  = fn(dist_fn(y_right))

weighted = (n_l/n)*H_left + (n_r/n)*H_right
return H_parent - weighted

# Trouver le meilleur seuil pour Total_Trans_Ct
feature = 'Total_Trans_Ct'
y_arr = y_tr.values
x_arr = X_tr[feature].values
thresholds = np.percentile(x_arr, np.arange(10, 100, 10))

print(f'=== GAIN D\'INFORMATION - {feature} ===')
best_gain, best_threshold = 0, None

for thr in thresholds:
    mask_left  = x_arr <= thr
    mask_right = x_arr > thr
    if mask_left.sum() < 5 or mask_right.sum() < 5:
        continue
    ig = gain_information(y_arr, y_arr[mask_left], y_arr[mask_right])
    print(f'  Seuil {thr:6.1f} | Gauche={mask_left.sum():5d} |
Droite={mask_right.sum():5d} | IG={ig:.5f}')
    if ig > best_gain:
        best_gain, best_threshold = ig, thr

print(f'Meilleur split : {feature} ≤ {best_threshold:.1f} (IG = {best_gain:.5f})')
```

Questions d'analyse

4. Quel seuil maximise le gain d'information pour **Total_Trans_Ct** ? Comparez ce seuil avec la distribution que vous avez observée dans l'EDA (TD3).
5. Répétez le calcul pour **Months_Inactive_12_mon**. Laquelle des deux features offre le meilleur gain d'information au premier split ?
6. **Réflexion** : pourquoi l'algorithme CART choisit-il le split qui **maximise** le gain d'information (et non le minimise) ?



Partie 2 — Arbre de Décision avec Sklearn

Durée estimée : 45 minutes | Construction, visualisation, diagnostic du sur-apprentissage et élagage.

Q 2.1

Premier arbre sans contrainte — diagnostic du sur-apprentissage

Consigne

Entraînez un arbre sans limitation de profondeur, puis comparez les scores train/test pour diagnostiquer le sur-apprentissage.

```
# Arbre sans contrainte (profondeur maximale)
clf_full = DecisionTreeClassifier(random_state=42)
clf_full.fit(X_tr, y_tr)

score_tr = clf_full.score(X_tr, y_tr)
score_te = clf_full.score(X_te, y_te)
f1_tr = f1_score(y_tr, clf_full.predict(X_tr))
f1_te = f1_score(y_te, clf_full.predict(X_te))

print('=== ARBRE SANS CONTRAINTE ===')
print(f'Profondeur réelle      : {clf_full.get_depth()}')
print(f'Nombre de feuilles     : {clf_full.get_n_leaves()}')
print(f'Accuracy train/test    : {score_tr:.4f} / {score_te:.4f}')
print(f'F1-score train/test    : {f1_tr:.4f} / {f1_te:.4f}')
print(f'→ Gap (sur-appr.)      : {score_tr - score_te:.4f}')

# Afficher les 3 premiers niveaux de l'arbre en texte
print('\n=== STRUCTURE (3 premiers niveaux) ===')
print(export_text(clf_full, feature_names=list(X_tr.columns),
                  max_depth=3))
```

⚠ Sur-apprentissage — Comment le détecter

- **Score train = 1.0** et **score test << 1.0** → sur-apprentissage évident
- L'arbre a mémorisé chaque exemple d'entraînement (feuilles avec 1-2 exemples)
- Le **gap** (train-test) > 0.05 est généralement significatif → régulariser
- Solution : limiter **max_depth**, augmenter **min_samples_leaf**, ou utiliser un ensemble (Random Forest)

Questions d'analyse



7. Quelle est la profondeur de l'arbre sans contrainte ? Est-ce raisonnable pour 10 127 exemples et ~20 features ?
8. Le gap accuracy train-test est-il important ? Comment qualifieriez-vous ce modèle en termes de biais/variance ?
9. En lisant les 3 premiers niveaux de l'arbre (export_text), quelle feature est utilisée pour le premier split ? Est-ce cohérent avec les résultats de la Partie 1 (gain d'information) ?

Q 2.2

Visualisation de l'arbre avec matplotlib et analyse

Consigne

Entraînez un arbre contraint (max_depth=4) et visualisez-le. Analysez les règles de décision générées.

```
# Arbre avec profondeur limitée (interprétable !)  
clf_viz = DecisionTreeClassifier(max_depth=4, min_samples_leaf=20,  
                                random_state=42)  
  
clf_viz.fit(X_tr, y_tr)  
  
print(f'Accuracy test (depth=4) : {clf_viz.score(X_te, y_te):.4f}')  
print(f'F1-score test          : {f1_score(y_te, clf_viz.predict(X_te)):.4f}')  
  
# Visualisation matplotlib (figsize très large pour lisibilité)  
fig, ax = plt.subplots(figsize=(26, 12))  
plot_tree(  
    clf_viz,  
    feature_names=list(X_tr.columns),  
    class_names=['Existing', 'Attrited'],  
    filled=True,  
    rounded=True,  
    fontsize=9,  
    ax=ax,  
    impurity=True,  
    precision=3  
)  
plt.title('Arbre de Décision - max_depth=4, min_samples_leaf=20',  
          fontsize=14, fontweight='bold')  
plt.tight_layout()  
plt.savefig('decision_tree_d4.png', dpi=150, bbox_inches='tight')  
plt.show()  
  
# Représentation texte complète  
print('\n=== ARBRE COMPLET (texte) ===')  
print(export_text(clf_viz, feature_names=list(X_tr.columns)))
```



i Comment lire un arbre visualisé sklearn

- Chaque **nœud interne** montre : feature, seuil, impureté (Gini), nombre d'exemples (samples), distribution des classes (value)
- Les **feuilles** sont les nœuds sans enfants — c'est là que la décision finale est prise
- La **couleur** reflète la classe majoritaire : orange = Attrited, bleu = Existing (selon palette filled=True)
- **Plus la couleur est saturée** → plus le nœud est pur (proche d'une seule classe)
- **Lire une règle** : suivre le chemin de la racine à une feuille — chaque arrête est une condition

Questions d'analyse

10. Quelle règle de décision simple (chemin de la racine à la feuille la plus pure) permet d'identifier les clients Attrited avec le plus de confiance ?
11. Un client avec **Total_Trans_Ct > 80** et **Total_Revolving_Bal > 1500** : suivez son chemin dans l'arbre. À quelle classe est-il classifié ?
12. La visualisation de l'arbre avec **max_depth=4** est-elle lisible ? À partir de quelle profondeur l'arbre devient-il illisible ? Quelle alternative utiliseriez-vous pour explorer un arbre profond ?

Q 2.3

Effet du max_depth — Courbe de validation

Consigne

Tracez la courbe de validation pour **max_depth** de 1 à 20. Identifiez la profondeur optimale et visualisez les 3 zones : underfitting, optimal, overfitting.

```
from sklearn.model_selection import validation_curve

# Courbe de validation : F1-score vs max_depth
depths = range(1, 21)
train_scores, val_scores = validation_curve(
    DecisionTreeClassifier(random_state=42, min_samples_leaf=10),
    X_tr, y_tr,
    param_name='max_depth',
    param_range=depths,
    cv=5,
    scoring='f1',
    n_jobs=-1
)

fig, ax = plt.subplots(figsize=(12, 6))
```



```
# Courbes moyennes
train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
val_mean = val_scores.mean(axis=1)
val_std = val_scores.std(axis=1)

ax.plot(depths, train_mean, 'o-', color='#1A7A6E', lw=2.5, label='F1 train')
ax.plot(depths, val_mean, 's-', color='#9C3D38', lw=2.5, label='F1 validation CV')
ax.fill_between(depths, train_mean-train_std, train_mean+train_std, alpha=.15,
color='#1A7A6E')
ax.fill_between(depths, val_mean-val_std, val_mean+val_std, alpha=.15,
color='#9C3D38')

# Profondeur optimale
opt_depth = list(depths)[np.argmax(val_mean)]
ax.axvline(opt_depth, color='#B07D3A', ls='--', lw=2,
label=f'Profondeur optimale = {opt_depth}')

# Zones
ax.axvspan(1, 3, alpha=.05, color='#9C3D38', label='Underfitting')
ax.axvspan(opt_depth-1, opt_depth+1, alpha=.1, color='#B07D3A', label='Zone
optimale')
ax.axvspan(opt_depth+2, 20, alpha=.05, color='#1A7A6E', label='Overfitting')

ax.set_xlabel('max_depth', fontsize=12)
ax.set_ylabel('F1-Score', fontsize=12)
ax.set_title('Courbe de Validation – Arbre de Décision', fontweight='bold',
fontsize=13)
ax.legend() ; plt.tight_layout() ; plt.show()

print(f'Profondeur optimale : {opt_depth}')
print(f'F1 val max : {val_mean.max():.4f}')
```

Questions d'analyse

13. À quelle profondeur les courbes train et validation divergent-elles significativement ? C'est le point où l'over-fitting commence.
14. La courbe de validation forme-t-elle un plateau après un certain point ? Qu'est-ce que cela implique pour le choix de max_depth ?
15. Quel est le F1-score maximal obtenu en cross-validation ? Par rapport au F1 de la Régression Logistique (TD4), l'arbre est-il meilleur ?

Q 2.4

Feature Importance — L'arbre comme sélecteur de features



```
# Entraîner avec la profondeur optimale
clf_opt = DecisionTreeClassifier(max_depth=opt_depth,
                                min_samples_leaf=10,
                                random_state=42)

clf_opt.fit(X_tr, y_tr)

# Feature importance (Gini) – normalisée
feat_imp = pd.Series(clf_opt.feature_importances_,
                     index=X_tr.columns).sort_values(ascending=False)

# Visualisation
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Top 15 features
feat_imp.head(15)[::-1].plot(kind='barh', ax=axes[0],
                             color='#1A2550', edgecolor='white')
axes[0].set_title('Feature Importance – Gini (top 15)', fontweight='bold')
axes[0].set_xlabel('Importance relative')

# Features avec importance nulle
zero_imp = feat_imp[feat_imp == 0]
print(f'Features jamais utilisées par l\'arbre : {len(zero_imp)}')
print(zero_imp.index.tolist())

# Cumulative importance
cum_imp = feat_imp.cumsum()
n_90 = (cum_imp < 0.9).sum() + 1
axes[1].plot(range(1, len(feat_imp)+1), cum_imp.values,
              color='#5B3D8F', lw=2.5)
axes[1].axhline(0.9, color='#B07D3A', ls='--', label='90% importance cumulée')
axes[1].axvline(n_90, color='#9C3D38', ls='--',
                 label=f'{n_90} features suffisent')
axes[1].set_xlabel('Nombre de features (par ordre d\'importance)')
axes[1].set_ylabel('Importance cumulée')
axes[1].set_title('Importance Cumulée des Features', fontweight='bold')
axes[1].legend()
plt.tight_layout() ; plt.show()
```

Questions d'analyse

16. Quelles sont les 3 features les plus importantes selon l'arbre ? Sont-elles cohérentes avec les features discriminantes identifiées dans l'EDA (TD3) ?
17. Combien de features sont nécessaires pour capter 90% de l'importance cumulée ? Combien de features sont totalement ignorées (importance = 0) ?



18. **Attention à l'importance Gini** : elle peut être biaisée vers les features à forte cardinalité (nombreuses valeurs uniques). Quelle méthode alternative (vue en cours au module avancé) est plus fiable ?



Partie 3 — K-Nearest Neighbors (KNN)

Durée estimée : 40 minutes | Trouver K optimal, démontrer l'effet de la normalisation, analyser les limites.

Q 3.1

Impact de la normalisation — démonstration expérimentale

Consigne

Entraînez KNN avec K=5 sur données brutes puis normalisées. Comparez les résultats et expliquez pourquoi la normalisation est fondamentale pour KNN.

```
# KNN sur données BRUTES (sans normalisation)
knn_raw = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn_raw.fit(X_tr, y_tr)
y_pred_raw = knn_raw.predict(X_te)
f1_raw = f1_score(y_te, y_pred_raw)

# KNN sur données NORMALISÉES
pipe_knn = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5, metric='euclidean'))
])
pipe_knn.fit(X_tr, y_tr)
y_pred_norm = pipe_knn.predict(X_te)
f1_norm = f1_score(y_te, y_pred_norm)

print('== IMPACT DE LA NORMALISATION SUR KNN (K=5) ==')
print(f' F1-score sans normalisation : {f1_raw:.4f}')
print(f' F1-score avec normalisation : {f1_norm:.4f}')
print(f' Gain : +{f1_norm-f1_raw:.4f}')
print('=====')

# Comprendre pourquoi : variance des features
print('\n=== VARIANCE DES FEATURES (brutes vs normalisées) ===')
scaler_check = StandardScaler()
X_tr_scaled = scaler_check.fit_transform(X_tr)

variance_raw = pd.Series(X_tr.var(), index=X_tr.columns)
variance_norm = pd.Series(X_tr_scaled.var(axis=0), index=X_tr.columns)

comparison = pd.DataFrame({'Variance brute': variance_raw.round(2),
                           'Variance normalisée': variance_norm.round(4)})
comparison = comparison.sort_values('Variance brute', ascending=False)
```



```
print(comparison.head(8).to_string())
```

Rappel de cours — Pourquoi KNN exige la normalisation

- La distance euclidienne traite chaque feature de façon **égale en termes d'unité**
- Sans normalisation, **Credit_Limit** (std≈9000) **domine totalement** la distance par rapport à **Avg_Utilization_Ratio** (std≈0.3)
- Après normalisation, chaque feature contribue avec une **variance unitaire** → influence équilibrée
- Formule : $d(x,y) = \sqrt{\sum (x_i - y_i)^2}$ — si std(Credit_Limit)=9000, un écart de 1 sur Credit_Limit = 30 000× un écart de 1 sur Utilization_Ratio
- **Règle absolue** : toujours normaliser avant KNN, TOUJOURS fitté le scaler sur train uniquement

Questions d'analyse

19. Quelle est la différence de F1-score entre KNN brut et normalisé ? Cette différence est-elle significative ?
20. Regardez le tableau variance brute vs normalisée. Quelle feature aurait écrasé toutes les autres dans le calcul de distance sans normalisation ?
21. Si vous utilisiez **Manhattan distance** (L1) plutôt qu'Euclidienne (L2), la normalisation serait-elle toujours nécessaire ? Pourquoi ?

Q 3.2

Courbe Elbow — Trouver K optimal

Consigne

Testez K de 1 à 30 et tracez la courbe Elbow (erreur = 1 - F1). Identifiez K optimal et expliquez le compromis biais/variance.

```
k_range = range(1, 31)
train_errors, val_errors = [], []

for k in k_range:
    pipe_k = Pipeline([
        ('scaler', StandardScaler()),
        ('knn', KNeighborsClassifier(n_neighbors=k))
    ])
    # Erreur sur le test set
    pipe_k.fit(X_tr, y_tr)
    val_errors.append(1 - f1_score(y_te, pipe_k.predict(X_te)))
```



```
train_errors.append(1 - fl_score(y_tr, pipe_k.predict(X_tr)))

opt_k = k_range[np.argmin(val_errors)]

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Courbe Elbow
axes[0].plot(k_range, train_errors, 'o-', color='#1A7A6E', lw=2, label='Erreur train')
axes[0].plot(k_range, val_errors, 's-', color='#9C3D38', lw=2, label='Erreur test')
axes[0].axvline(opt_k, color='#B07D3A', ls='--', lw=2,
                label=f'K optimal = {opt_k}')
axes[0].set_xlabel('K (nombre de voisins)') ; axes[0].set_ylabel('1 - F1 (Erreur)')
axes[0].set_title('Courbe Elbow - KNN', fontweight='bold')
axes[0].legend()

# Zoom sur la zone Underfitting / Overfitting
axes[1].fill_between(k_range, 0, [v-min(val_errors) for v in val_errors],
                    alpha=.3, color='#9C3D38')
axes[1].plot(k_range, [t-v for t,v in zip(train_errors,val_errors)],
            'o-', color='#5B3D8F', lw=2, label='Gap (train-val)')
axes[1].axhline(0, color='black', ls='-', lw=0.8)
axes[1].axvline(opt_k, color='#B07D3A', ls='--', lw=2)
axes[1].set_xlabel('K') ; axes[1].set_ylabel('Écart train-test')
axes[1].set_title('Variance du modèle (gap train-test)', fontweight='bold')
axes[1].legend()
plt.tight_layout() ; plt.show()

print(f'K optimal : {opt_k}')
print(f'F1-score test optimal : {1-min(val_errors):.4f}')
```

Rappel de cours — Comprendre la courbe Elbow — Biais/Variance pour KNN

- **K=1** : sur-apprentissage maximal (le modèle mémorise les données d'entraînement, erreur train = 0)
- **K grand** : sous-apprentissage — frontière trop lisse, ne capte plus les patterns fins
- **K optimal** : le coude de la courbe où l'erreur test est minimale
- **Gap train-test** qui diminue quand K augmente → cohérent avec le bias-variance tradeoff
- **Règle pratique** : tester K impair pour les classifications binaires (évite les égalités)

Questions d'analyse

22. À K=1, pourquoi l'erreur d'entraînement est-elle exactement 0 ? Est-ce un bon signe ?
23. À partir de quelle valeur de K l'erreur test commence-t-elle à remonter (underfitting) ?



24. Le K optimal est-il pair ou impair ? Y a-t-il une raison de préférer les valeurs impaires pour la classification binaire ?
25. Comparez le F1-score de KNN optimal avec celui de l'arbre de décision optimal (Q2.3). Lequel est meilleur sur ce dataset ?

Q 3.3

Visualisation de la frontière de décision KNN — données 2D

Consigne

Réduisez à 2 features pour visualiser et comparer les frontières de décision KNN (K=1, K optimal, K=20) dans un plan 2D.

```
feat1, feat2 = 'Total_Trans_Ct', 'Total_Revolving_Bal'
X_2d = data[[feat1, feat2]].values
y_2d = data['churn'].values

X_2d_tr, X_2d_te, y_2d_tr, y_2d_te = train_test_split(
    X_2d, y_2d, test_size=0.2, random_state=42, stratify=y_2d)

K_vals = [1, opt_k, 20]
fig, axes = plt.subplots(1, 3, figsize=(19, 6))

for ax, k in zip(axes, K_vals):
    pipe_2d = Pipeline([
        ('sc', StandardScaler()),
        ('knn', KNeighborsClassifier(n_neighbors=k))
    ])
    pipe_2d.fit(X_2d_tr, y_2d_tr)

    # Grille
    scaler_2d = pipe_2d.named_steps['sc']
    x_min, x_max = X_2d[:,0].min()-5, X_2d[:,0].max()+5
    y_min, y_max = X_2d[:,1].min()-100, X_2d[:,1].max()+100
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                          np.linspace(y_min, y_max, 200))
    Z = pipe_2d.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:,1]
    Z = Z.reshape(xx.shape)

    ax.contourf(xx, yy, Z, levels=20, cmap='RdYlGn_r', alpha=.55)
    ax.contour(xx, yy, Z, levels=[0.5], colors='navy', linewidths=2)

    for cls, color in [(0, '#1A7A6E'), (1, '#9C3D38')]:
        mask = y_2d_te==cls
        ax.scatter(X_2d_te[mask,0], X_2d_te[mask,1],
```



```
c=color, alpha=.5, s=10)

f1 = f1_score(y_2d_te, pipe_2d.predict(X_2d_te))
ax.set_title(f'KNN K={k} (F1={f1:.3f})', fontweight='bold')
ax.set_xlabel(feat1) ; ax.set_ylabel(feat2) if k==K_vals[0] else None

plt.suptitle('Frontières de Décision KNN – K=1 / K optimal / K=20',
             fontsize=13, fontweight='bold')
plt.tight_layout() ; plt.show()
```

Questions d'analyse

26. La frontière de décision de K=1 est-elle régulière ou très irrégulière ? Qu'est-ce que cela révèle sur ce modèle ?
27. Comment la frontière évolue-t-elle entre K=1 et K=20 ? Pourquoi dit-on que K est un hyperparamètre de **lissage** ?
28. La frontière de KNN est-elle linéaire ou non-linéaire ? Comparez avec la frontière de la Régression Logistique (TD4).



Partie 4 — Comparaison et Analyse Comparative

Durée estimée : 25 minutes | Comparer rigoureusement Arbre de Décision vs KNN — forces, limites et cas d'usage.

Q 4.1

Benchmark complet — Arbre vs KNN vs Régression Logistique

```
from sklearn.model_selection import cross_val_score

# Modèles optimisés
models = {
    'Arbre (depth optimal)': Pipeline([
        ('m', DecisionTreeClassifier(max_depth=opt_depth,
                                     min_samples_leaf=10, random_state=42))
    ]),
    'KNN (K optimal)': Pipeline([
        ('sc', StandardScaler()),
        ('m', KNeighborsClassifier(n_neighbors=opt_k))
    ]),
    'Régression Logistique': Pipeline([
        ('sc', StandardScaler()),
        ('m', __import__('sklearn.linear_model',
fromlist=['LogisticRegression']).LogisticRegression(
            max_iter=1000, random_state=42))
    ]),
}

print('===== BENCHMARK - 3 MODÈLES (5-Fold CV, scoring=F1) =====')
results = {}
for name, model in models.items():
    scores = cross_val_score(model, X_tr, y_tr, cv=5, scoring='f1', n_jobs=-1)
    # Test final
    model.fit(X_tr, y_tr)
    y_p = model.predict(X_te)
    f1_te_val = f1_score(y_te, y_p)
    results[name] = {
        'F1 CV (moy)': round(scores.mean(), 4),
        'F1 CV (std)': round(scores.std(), 4),
        'F1 Test': round(f1_te_val, 4),
    }
    print(f' {name:30s} CV={scores.mean():.4f}±{scores.std():.4f}
Test={f1_te_val:.4f}')
print('=====')

# Matrice de confusion côte à côte
fig, axes = plt.subplots(1, 3, figsize=(17, 5))
```



```
for ax, (name, model) in zip(axes, models.items()):
    model.fit(X_tr, y_tr)
    cm = confusion_matrix(y_te, model.predict(X_te))
    ConfusionMatrixDisplay(cm, display_labels=['Existing', 'Attrited']).plot(ax=ax,
    colorbar=False, cmap='Blues')
    ax.set_title(f'{name}', fontweight='bold', fontsize=11)
plt.suptitle('Matrices de Confusion – Comparaison des 3 Modèles', fontweight='bold',
    fontsize=13)
plt.tight_layout() ; plt.show()
```

Questions d'analyse

29. Quel modèle obtient le meilleur F1-score sur le test set ? La différence est-elle statistiquement significative (comparez les écarts-types de CV) ?
30. Quel modèle a le plus grand écart entre F1 CV et F1 Test ? Qu'est-ce que cela signifie ?
31. En comparant les matrices de confusion, quel modèle détecte le mieux la classe Attrited (Rappel de la classe 1) ?

Q 4.2

Analyse : quand utiliser Arbre vs KNN ?

Critère de choix	Arbre de Décision	KNN
Interprétabilité requise	✓ Excellent — règles lisibles	X Boîte noire totale
Features mixtes (num+cat)	✓ Gère nativement	X Requiert encodage + norm.
Dataset très grand (>100K)	✓ Entraîn. rapide	X Prédiction lente $O(n \cdot d)$
Haute dimensionnalité	Dégrade (overfitting)	X Malédiction dimensionnalité
Relations non-linéaires	✓ Capture nativement	✓ Capture sans hypothèse
Baseline rapide	✓ Excellent baseline	✓ Bon baseline
Nécessite normalisation	X Non	✓ Obligatoire
Point fort en production	Règles métier explicables	Détection d'anomalies proches

Questions d'analyse

32. Un directeur de banque vous demande d'expliquer pourquoi un client a été classé 'à risque de churn'. Quel modèle choisissez-vous ? Pourquoi ?
33. Vous devez déployer un modèle de scoring en temps réel pour 10 000 requêtes/seconde. Quel modèle exclure d'emblée ? Justifiez avec la complexité algorithmique.



34. Dans un contexte RGPD (droit à l'explication), la Régression Logistique est souvent préférée à KNN en production. Pourtant, un arbre de décision est encore plus facile à expliquer. Quel compromis feriez-vous en pratique ?



Section Bonus — Pour aller plus loin

Ces exercices approfondissent deux aspects avancés : l'élagage post-hoc de l'arbre et la malédiction de la dimensionnalité.

Bonus B1 — Élagage Cost-Complexity (ccp_alpha)

sklearn offre un élagage automatique basé sur le paramètre `ccp_alpha` : plus `ccp_alpha` est grand, plus l'arbre est élagué.

```
# Chemin de coût-complexité
path = DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(X_tr,
y_tr)
ccp_alphas, impurities = path.ccp_alphas, path.impurities

# F1 pour chaque valeur d'alpha
f1_train_list, f1_test_list = [], []
for alpha in ccp_alphas[::-5]: # Sous-échantillonner pour accélérer
    clf_pruned = DecisionTreeClassifier(ccp_alpha=alpha, random_state=42)
    clf_pruned.fit(X_tr, y_tr)
    f1_train_list.append(f1_score(y_tr, clf_pruned.predict(X_tr)))
    f1_test_list.append(f1_score(y_te, clf_pruned.predict(X_te)))

plt.figure(figsize=(10, 5))
plt.plot(ccp_alphas[::-5], f1_train_list, 'o-', label='F1 train', color='#1A7A6E')
plt.plot(ccp_alphas[::-5], f1_test_list, 's-', label='F1 test', color='#9C3D38')
plt.xlabel('ccp_alpha (plus grand = plus d\'élagage)')
plt.ylabel('F1-Score') ; plt.legend()
plt.title('Élagage Cost-Complexity', fontweight='bold')
plt.show()
```

35. Quel `ccp_alpha` donne le meilleur F1 test ? L'arbre résultant est-il plus simple que celui obtenu avec `max_depth` ?

Bonus B2 — Malédiction de la dimensionnalité

Démontrez expérimentalement que KNN perd en efficacité quand le nombre de features augmente.

```
# Générer des données synthétiques avec dimensions croissantes
from sklearn.datasets import make_classification

dims = [2, 5, 10, 20, 50, 100, 200]
f1_scores = []

for n_features in dims:
```



```
X_syn, y_syn = make_classification(
    n_samples=1000, n_features=n_features,
    n_informative=max(2, n_features//4),
    n_redundant=0, random_state=42
)
X_s_tr, X_s_te, y_s_tr, y_s_te = train_test_split(
    X_syn, y_syn, test_size=0.2, random_state=42)
pipe_d =
Pipeline([('sc',StandardScaler()),('knn',KNeighborsClassifier(n_neighbors=5))])
pipe_d.fit(X_s_tr, y_s_tr)
f1 = f1_score(y_s_te, pipe_d.predict(X_s_te))
f1_scores.append(f1)
print(f'  dim={n_features:4d}  F1={f1:.4f}')
```



```
plt.figure(figsize=(10, 5))
plt.plot(dims, f1_scores, 'o-', color='#5B3D8F', lw=2.5)
plt.xlabel('Nombre de features (dimensions)') ; plt.ylabel('F1-Score KNN')
plt.title('Malédiction de la dimensionnalité – KNN (K=5)', fontweight='bold')
plt.xscale('log') ; plt.show()
```

36. Le F1-score de KNN diminue-t-il régulièrement avec le nombre de dimensions ? À partir de combien de features la dégradation devient-elle significative ?
37. Expliquez intuitivement pourquoi KNN se dégrade en haute dimension en termes de distances (hint : que se passe-t-il avec la distance min et max quand $n_features \rightarrow \infty$) ?



Livrable attendu

✓ Ce que vous devez remettre

- Notebook **TD5_NomPrenom.ipynb** — toutes les cellules exécutées avec outputs visibles
- **Calculs manuels** d'entropie/Gini vérifiés avec NumPy et commentés
- **Visualisation de l'arbre** (max_depth=4) avec interprétation d'au moins 2 règles de décision
- **Courbe de validation** max_depth avec identification des zones under/over-fitting
- **Courbe Elbow** KNN avec K optimal justifié et frontières de décision comparées
- **Benchmark** des 3 modèles (Arbre + KNN + Logistique) avec tableau comparatif
- Une cellule Markdown de **conclusion** (~200 mots) : Arbre vs KNN — quand utiliser lequel, leçons tirées
- Délai : **avant le TD 6**, dépôt sur la plateforme pédagogique

Critères d'évaluation

Critère	Coefficient	Description
Calculs manuels entropie/Gini (exactitude + formules)	20%	Vérification NumPy, explication formule
Visualisation + interprétation de l'arbre	20%	Règles lisibles, lien avec EDA
Courbes de validation et Elbow — diagnostic biais/variance	25%	Zones identifiées, paramètres optimaux
Benchmark comparatif 3 modèles	25%	Tableau, matrices confusion, justifications
Section Bonus (optionnel)	+ 10%	Élagage ccp_alpha ou malédiction dimensionnalité

Ressources et Documentation

- **sklearn — DecisionTreeClassifier** : <https://scikit-learn.org/stable/modules/tree.html>
- **sklearn — KNeighborsClassifier** : <https://scikit-learn.org/stable/modules/neighbors.html>
- **Visualisation d'arbres (graphviz)** : https://scikit-learn.org/stable/modules/generated/sklearn.tree.export_graphviz.html
- **ID3 — Quinlan 1986** : <https://link.springer.com/article/10.1007/BF00116251>



- **sklearn — validation_curve** : https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.validation_curve.html

Installer graphviz pour la visualisation d'arbres

```
# Installation système (Ubuntu/Debian)
sudo apt-get install graphviz

# Installation Python
pip install graphviz pydotplus

# Export et visualisation d'arbre
from sklearn.tree import export_graphviz
import graphviz

dot_data = export_graphviz(
    clf_opt, feature_names=X_tr.columns,
    class_names=['Existing', 'Attrited'],
    filled=True, rounded=True
)
graphviz.Source(dot_data) # Affiche l'arbre dans Jupyter
```